DynaMoDe-NeRF: Motion-aware Deblurring Neural Radiance Field for Dynamic Scenes

Ashish Kumar Rajagopalan A. N.
Indian Institute of Technology Madras, India
ee20d006@smail.iitm.ac.in, raju@iitm.ac.in

Abstract

Neural Radiance Fields (NeRFs) have made significant advances in rendering novel photorealistic views for both static and dynamic scenes. However, most prior works assume ideal conditions of artifact-free visual inputs i.e., images and videos. In real scenarios, artifacts such as object motion blur, camera motion blur, or lens defocus blur are ubiquitous. Some recent studies have explored novel view synthesis using blurred input frames by examining either camera motion blur, defocus blur, or both. However, these studies are limited to static scenes. In this work, we enable NeRFs to deal with object motion blur whose local nature stems from the interplay between object velocity and camera exposure time. Often, the object motion is unknown and time varying, and this adds to the complexity of scene reconstruction. Sports videos are a prime example of how rapid object motion can significantly degrade video quality for static cameras by introducing motion blur. We present an approach for realizing motion blur-free novel views of dynamic scenes from input videos with object motion blur captured from static cameras spanning multiple poses. We propose a NeRF-based analytical framework that elegantly correlates object three-dimensional (3D) motion across views as well as time to the observed blurry videos. Our proposed method DynaMoDe-NeRF (Dynamic Motion-aware Deblurring NeRF) is self-supervised and reconstructs the dynamic 3D scene, renders sharp novel views by blind deblurring, and recovers the underlying 3D motion and blur parameters. We provide comprehensive experimental analysis on synthetic and real data to validate our approach. To the best of our knowledge, this is the first work to address localized object motion blur in the NeRF domain. The dataset is available at: https://github.com/akumar005/DynaMoDe-NeRF

1. Introduction

Neural Radiance Field (NeRF) [25] is capable of synthesizing novel views of static scenes [25, 35, 45, 47, 51, 56] and dynamic scenes [19, 20, 30, 33, 40, 46, 57] with high fidelity. While NeRF excels in synthesizing static scenes, its application to dynamic scenes, where objects are in motion, poses unique challenges. Unlike static scenes, which maintain sufficient overlap and visual redundancy across frames, dynamic scenes are inherently sparse and this warrants integration of object motion or features across temporal dimension to faithfully learn a dynamic scene representation.

To share information across time while respecting temporal coherence, works such as [31, 33, 46, 57] learn a deformation field. This maps the scene to a canonical space, which is then used to estimate radiance and density. Though this allows the maintenance of temporal coherence, these approaches are effective for subjects with relatively small motion. Neural Scene Flow Fields, NSFF [19] learn the scene flow in 3D to enforce consistency across time instances. The 3D scene flow is guided by the 2D motion field (optical flow) which is estimated from an off-the-shelf state-of-the-art network. These methods additionally leverage depth maps as a supervision signal to enforce geometric consistency. [20] exploits image-based rendering (IBR) to learn dynamic scenes by relying on motion masks, optical flow, and depth as auxiliary signals. IBR exploits features from nearby frames to maintain spatio-temporal consistency and has been shown to work for long videos. However, when adjacent frames are corrupted (e.g., due to blur), the extracted features become noisy and unreliable, thus compromising the integrity of the features and hindering accurate reconstruction. Explicit neural representation-based methods [2, 8] use a grid-based or voxel-based approach to share features along the spatial and temporal dimensions. These methods can generate photorealistic novel views. But, they assume ideal, artifact-free visual inputs. In practice, image artifacts such as camera motion blur, object motion blur, or defocus blur are almost unavoidable. An artifact-corrupted input can severely affect rendering quality and poses significant challenges for applications that rely on high-quality dynamic scene representations.

Only recently have researchers begun addressing the challenge of novel view synthesis from artifact-corrupted views, with particular attention to blurring artifacts [15, 16, 24, 32, 49]. Methods, such as [4, 38, 59] exploit Gaussian splat-

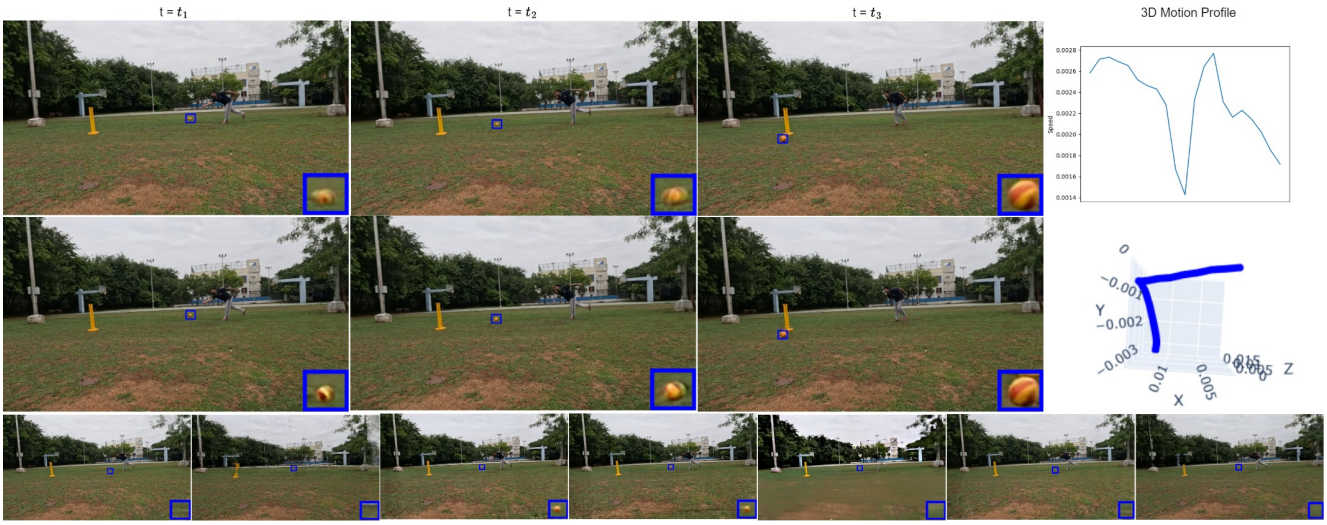


Figure 1. Our proposed DynaMoDe-NeRF uses posed multi-view videos of dynamic scenes with object motion blur to generate sharp novel views and recover 3D motion (velocity and trajectory). The first row shows blurry frames from a test viewpoint, while the second row shows corresponding rendered sharp novel views at times t_1, t_2, and t_3. Our method effectively handles varying blur levels, from minor (column 3 at t_3) to significant (columns 1 and 2). The last column illustrates the recovered 3D motion (speed and 3D trajectory) effectively capturing the physics. Competing works, Hexplane [2], Mixvoxel [48], Deblur-NeRF [24], DP-NeRF [16], DyBluRF [44], GS4D [52], and KPlanes [10] (from left to right in the last row), fail to synthesize novel sharp views in a robust manner (shown for time t_1). Some miss the moving object completely.

ting to model blur. However, all these works focus on either camera motion blur, defocus blur, or both, targeting static scenes. These methods model blur either by learning camera motion during the exposure time, learning blur kernel locations and weights, or using priors such as sharpness prior to model blur in the scene with a limited number of possible kernels. Apart from being curated for static scenes, these methods additionally assume the entire image to be blurred or have a sharpness score. On the other hand, object motion-induced blur is local.

In this work, we propose Dynamic Motion-aware Deblurring NeRF (DynaMoDe-NeRF) to tackle the challenge of novel view synthesis in the presence of object motion blur which is a highly prevalent artifact. This arises when the camera fails sharply capture a moving object within its exposure interval. Although reducing the exposure duration can alleviate blur, it often leads to other issues such as low-light artifacts and photon noise. Moreover, object velocity is typically unknown in most real-world scenarios. Compounding the problem, an object motion path is often complex, with varying speeds that result in temporally varying blur across video frames. The severity of this motion-induced blur is further influenced by depth, leading to variations across different views. Notably, even in a single-view setup, maintaining temporal consistency of the restored frames is crucial. In the multi-view context, the problem becomes even more intricate due to the added requirement of ensuring view-consistency across multiple perspectives. In such a scenario, achieving robust, faithful, and consistent estimation of fine-grained feature-based auxiliary signals such as optical flow and depth, which are commonly used as supervisory signals for dynamic novel view synthesis becomes challenging, resulting in noisy and inaccurate outputs. Thus, it is crucial to develop methods to handle motion blur effectively.

Our method takes blurry videos of a dynamic scene captured from multiple views as inputs, reconstructs the sharp scene, renders novel views, and recovers the 3D motion profile of the moving object as shown in Fig. 1. We utilize an object motion mask, which does not depend on finer details such as features as in optical flow or depth estimation, as an additional input to guide the learning process. Our deblurring strategy is grounded on object 3D motion and the principles of blind deblurring. We meticulously establish and harness the relation between 3D motion and the induced continuous blur kernel. This involves mapping 3D object motion to the perceived 2D motion across multiple views simultaneously and extends this correlation across different time instances for each view. Doing so ensures a consistent and coherent representation of dynamic scenes, capturing spatial, temporal, and view dependencies as well. Instead of learning 3D velocities for all time steps M, we effectively learn a reduced set of L control velocities ($L \ll M$) and use linear interpolation to estimate velocities at any specific time step. This mitigates the computational complexity while still accurately capturing the underlying scene dy-

namics.

In addition to utilizing the analytical framework for calculating continuous blur kernel locations, we introduce a depth and velocity-aware network called BlurKernelNet, which is a three-layer perception (MLP) designed to learn the weights associated with the blur kernel locations. The weighted sum of sharp intensities at the kernel locations is computed so as to match the corresponding blurry observation.

To summarize, our main contributions are the following:

- Dynamic object deblurring in NeRF which is a first work.
- We propose an analytical framework that correlates 3D motion to 2D continuous blur kernel locations across time and camera views.
- We devise BlurKernelNet, a 3-layer MLP network, that learns kernel weights in a velocity and depth-aware manner by encoding depth and motion dependencies of blur.
- We introduce a 3D motion smoothness regularizer that respects temporally smooth transition of motion.
- Through extensive experiments on synthetic data, we demonstrate superior deblurring quality and accurate recovery of 3D motion profiles, validating the effectiveness of our approach. Additionally, we provide qualitative results on real-world data, further showcasing the robustness of our method.

2. Related Works

Dynamic NeRF: NeRF [25] takes a $5D$ input (X, Y, Z, θ, ϕ), where $[X, Y, Z]^T$ is a scene point and (θ, ϕ) represent viewing direction, encoding the view-dependent color, and models the continuous 3D scene by implicit neural representation. Dynamic NeRF deals with scenes that have a background (white or textured) and a foreground, such as a human or an object in motion. Naively extending the input to a 6D signal with time t as an extra dimension severely affects performance [33]. Several works have been proposed to render novel views for dynamic scenes [1, 19, 20, 22, 23, 30, 31, 33, 46]. Most of the neural scene representation-based rendering approaches consider one of two methods; the first models the scene by learning a canonical space and a deformation field. The observation space is transformed to a canonical space for color and density estimation based on the learned deformation field. But this approach often struggles in the case of significant motion. The second approach directly learns the dynamic scene by learning the motion field or by exploiting features from nearby frames that capture temporal information across the observed frames. While achieving photorealism in rendered views for scenes with significant object motion, these methods, however, rely on several auxiliary signals such as optical flow, depth map, and motion masks as supervisory signals to learn the scene motion. Hence, their performance and rendered view quality are determined by the accuracy and reliability of these auxiliary signals. Explicit frameworks for dynamic scene representation often utilize voxels [8, 11, 21, 48] or grids [2, 34, 40]. These approaches leverage shared features across time to learn temporally varying scenes. However, all these methods assume ideal, artifact-free inputs which is unrealistic.

NeRF Based Deblurring: Blind image deblurring is a classical ill-posed problem that involves estimating the blur kernel and the latent (sharp) image given a single blurred observation. Several traditional [5, 9, 13, 18, 29, 39, 55] and deep learning based methods [3, 14, 17, 27, 28, 41–43, 50, 53, 54] have been proposed in the past. In the NeRF domain many recent methods [7, 15, 16, 24, 32, 49] have attempted the task of novel view synthesis from blurred inputs. [24], inspired by image-based $2D$ convolution, addresses the problem of camera motion blur and defocus blur in NeRF. It learns sparse kernel locations and weights to blur the latent sharp view to match the input views. [49] addressed blur caused by camera motion by modeling the camera trajectory with a Special Euclidean SE(3)motion model during exposure. Based on the optimized camera pose during exposure time, they render the scene and average the rendered images to match the input blurry image. [15] addresses defocus blur artifacts by using a sharpness prior to estimate the defocus map and quantizing it to L levels and equivalently optimizing for L blur kernels. [32] proposed Progressively Deblurring Radiance Field (PDRF) and employs a Progressive Blur Estimation (PBE) module by combining 2D and 3D features to refine blur modeling to address camera motion and defocus blur views. [16] addresses defocus blur and camera motion blur artifacts. It models blurring with SE(3) field relying on physical priors. [44] introduced a monocular video deblurring method that assumes the entire scene is blurred. However, it does not specifically address foreground deblurring and the recovery of 3D motion.

3. Methodology

Given a set of posed multiview videos of dynamic scenes corrupted by object motion blur, our goal is to (i) *render sharp 4D spatio-temporal novel views,* (ii) *recover object 3D motion profile (speed and trajectory), and* (iii) *decipher blur kernel.* The core of our approach lies in correlating object motion with the underlying physics of blur formation.

We begin with a brief overview of NeRF [25] and HexPlane [2], which form the basis of our method. In a reference coordinate frame, for a particular view v, NeRF samples 3D points, $\underline{x}_i \in \mathbb{R}^3$, where $i = 1, 2, \ldots, N$ along a ray $\underline{r}(\underline{y}_r) \in \mathbb{R}^3$ originating from the camera center $\underline{o}_c \in \mathbb{R}^3$, and passing through the pixel $\underline{y}_r \in \mathbb{R}^2$. The points $\underline{x}_i = \underline{o}_c + t_i \cdot \underline{r}(\underline{y}_r)$ are sampled within the depth bounds provided by a pose-estimation framework, often based on structure-from-motion (SfM) techniques like COLMAP [36, 37]. t_i is the sampling step size. Each $\underline{x}_i$ is processed by a multilayer perceptron (MLP) to estimate its density $\sigma_i \in \mathbb{R}$, and

color $c_i \in [0,1]^3$. The color estimation is conditioned on the view direction to model view-dependent radiance. Let $\delta_i = t_{i+1} - t_i$ be the distance between two successive sampled points. In the absence of blur, the rendered color at a pixel $\underline{y}_r$ is given by

$$v^{sharp}(\underline{y}_r) = \sum_{i=1}^{N} T_i \left(1 - e^{-(\ \sigma_i \delta_i)}\right) c_i \tag{1}$$

where $T_i = e^{-\left(\sum_{j=1}^{i-1} \sigma_j \delta_j\right)}$. While several methods have been proposed to extend NeRF to dynamic scenes, we focus on grid-based approach that decomposes a 4D scene into six 2D planes [2]. Our framework builds upon this.

$$\begin{aligned} V_{4D} = &\sum_{r=1}^{R_1} P_r^{XY} \circ P_r^{tZ} \circ v_r^1 + \sum_{r=1}^{R_2} P_r^{YZ} \circ P_r^{tX} \circ v_r^2 \\ &+ \sum_{r=1}^{R_3} P_r^{XZ} \circ P_r^{tY} \circ v_r^3 \end{aligned} \tag{2}$$

Here $\circ$ represents outer product. P_r^{XY}, P_r^{YZ}, P_r^{ZX}, P_r^{tX}, P_r^{tY}, P_r^{tZ} are feature planes spanning the XY, YZ, ZX, tX, tY, tZ planes, respectively, and $\mathbf{v}_r^X, \mathbf{v}_r^Y, \mathbf{v}_r^Z$ represent vectors along the X, Y, Z axes. Additionally, $\mathbf{v}_r^1, \mathbf{v}_r^2, \mathbf{v}_r^3$ represent vectors along the feature axis. For a sampled point $\underline{x}_i$, the volume $\mathbf{V}_{4D}$ can be queried using bilinear interpolation to extract as $\underline{f}(\underline{x}_i) = V_{4D}(\underline{x}_i)$. The feature $\underline{f}(\underline{x}_i)$ is then provided as input to an MLP to estimate the color and density. The final color of the ray is rendered using Eqn. 1.

Blur due to a moving object occurs when multiple scene points are mapped to the same pixel location during exposure. From a 2D perspective, it is equivalent to mixing several sharp pixel color values onto a single pixel. Mathematically, the estimated blurred intensity at time t, for the v^{th} view at a pixel $\underline{y}_r^t$ on a ray $\underline{r}$ can be expressed as

$$v^{\text{blur}}(\underline{y}_r^t) = \sum_{\underline{y}_j^t \in \mathcal{N}(\underline{y}_r^t)} w_j \cdot v^{\text{sharp}}(\underline{y}_j^t) \tag{3}$$

where w_j is the blur kernel relating the association of latent sharp intensities to the observed blurry intensity. $\mathcal{N}(\underline{y}_r^t)$ represents the blur neighborhood for $\underline{y}_r^t$. In a practical scenario, w_j and $v^{sharp}(\underline{y}_j^t)$ are not known, and the challenge lies in estimating them. The blur kernel w_j controls the extent and severity of blur. It can vary within an image (spatially varying blur) as well as across views and is governed by the 3D shape of the moving object, its motion parameters, and the camera viewpoint. In a multiview setup, the key challenge is to ensure consistency across space, time, and views within the deblurring process.Our proposed method disentangles the blur kernel into two components: geometry (the spatial distribution of the kernel) and photometry (intensity mixing kernel weights), both on the image plane. Since we leverage object 3D motion, which is a

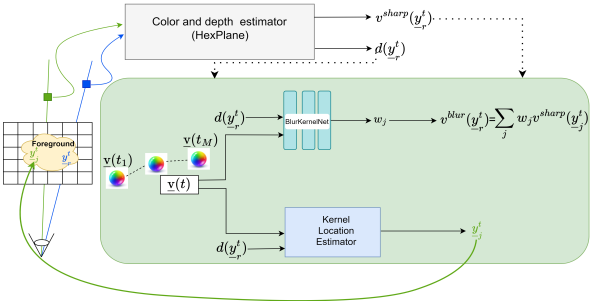


Figure 2. Overview of our approach: To render a blurred intensity at time t and view v, we analytically determine the continuous kernel locations and compute the latent sharp intensities using HexPlane [2] and linearly mix them with the estimated weights from BlurKernelNet. We jointly optimize the scene (HexPlane), object motion, and BlurKernelNet. During testing, the green highlighted section is discarded.

fundamental and global cue for blur formation across views during the exposure time, we are able to handle complex blur patterns, from high to negligible blur during the motion period as can be seen in Fig. 1.We jointly optimize for 3D scene representation i.e. HexPlane [2], object motion, and BlurKernelNet to offer a robust solution to the challenges of novel view synthesis and 3D motion recovery from motion-blurred multiview videos. We assume that the blur arises from the motion of a rigid body. A visualization of our approach is provided in Fig. 2.

3.1. Kernel Location Estimation (Geometry)

In our conceptual framework, we interpret a dynamic 3D object as a collection of well-structured atomic particles $\underline{o}_r^t \in \mathbb{R}^3, r = \{1, 2, \ldots\}$. Each $\underline{o}_r^t$ is distinguished by specific attributes, such as color, and adheres to a set of physical principles ensuring their cohesion within the object structure. We assume that these particles are spatially sampled by the camera such that, at any given time, only one $\underline{o}_r^t$ maps to the corresponding pixel $\underline{y}_r^t$. This relationship is given by $\underline{y}_r^t = P^v \underline{o}_r^t$, where P^v represents the 3×4 projection matrix for view v.

From a geometric perspective, blur can be interpreted as the result of a many-to-one mapping, as illustrated in Fig. 3 and is operative during the exposure period of a camera. We assume that a total of $2m + 1$ points are mapped to $\underline{y}_r^t$ during exposure to cause blurring. This process entails the projection of multiple $\underline{o}_j^t, j = 1, 2, \cdots 2m+1$ onto an identical 2D pixel location on the image sensor. Within this 2D plane, the projected attributes, typically through a weighted summation, yield the blurred color intensity.Conversely, a

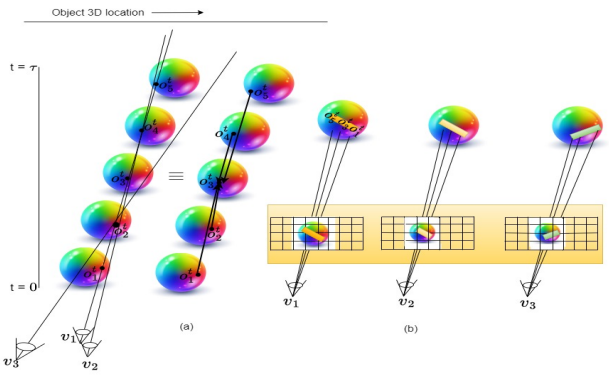


Figure 3. Illustration of blur formation during a single exposure τ across views v: Each view captures distinct 2D motion paths (yellow, (b)) and observes different object points. Points for view v_1 are shown here. Despite view differences, object 3D motion is consistent in a shared reference frame, with multiple 3D points ($\underline{o}_j^t$) mapping to a single pixel location during exposure, governed by object velocity. Similar poses (e.g., v_1, v_2) can produce varied blur due to depth. The rectangular patch shows the 2D blur kernel direction, indicating points affecting sharpness.

sharp image is characterized by a direct one-to-one correspondence between 3D and 2D spaces, wherein each pixel correlates to a singular attribute or color of its 3D counterpart. The inverse task of deducing $\underline{o}_j^t$ and its associated features from blurred observations poses a fundamentally ill-posed problem.Instead of directly regressing for $\underline{o}_j^t$ or the blur kernel locations $\underline{y}_j^t$ from a neural network, we implicitly estimate $\underline{o}_j^t$s, which are related to the latent sharp point $\underline{o}_r^t$ and map to $\underline{y}_j^t$s during exposure τ as

$$\underline{o}_j^t = \underline{o}_r^t + (j - (m+1))\,\frac{\tau}{2m}\underline{\mathrm{v}}(t) \tag{4}$$

This enforces multiview consistency and motion aware deblurring by treating motion as a global cue. It also enables continuous blur kernel spatial location estimation. In the equation above, we assume that for a rigid body in translational motion with velocity $\underline{\mathrm{v}} \in \mathbb{R}^3$ over exposure time τ, all surface points share the same instantaneous velocity.
Next, we find the kernel locations $\underline{y}_j^t$ by projecting these points onto the image plane as

$$\underline{y}_j^t = \Pi^{-1}\left(P^v\Pi(\underline{o}_r^t) + (j - (m+1))\,\frac{\tau}{2m}P^v\tilde{\Pi}(\underline{\mathrm{v}})\right) \tag{5}$$

We further simplify this by using the fact that $\underline{y}_r^t = P^v\underline{o}_r^t$ and the relation depends on explicit 2D quantities.

$$\underline{y}_j^t = \Pi^{-1}\left(d(\underline{\mathrm{y}}_r^t)\Pi(\underline{y}_r^t) + (j - (m+1))\,\frac{\tau}{2m}P_v\tilde{\Pi}(\underline{\mathrm{v}})\right) \tag{6}$$

where $d(\underline{y}_r^t)$ is the depth of $\underline{y}_r^t$, $\Pi : \mathbb{R}^3 \to \mathbb{R}^4$, $\Pi(\underline{y}) = [\underline{y}, 1]^{\mathrm{T}}$ is a homogenization function, $\tilde{\Pi} : \mathbb{R}^3 \to \mathbb{R}^4, \tilde{\Pi}(\underline{\mathrm{v}}) = [\underline{\mathrm{v}}, 0]^{\mathrm{T}}$ is modified homogenization function, $P_v : \mathbb{R}^4 \to \mathbb{R}^3$ is the projective mapping transformation for view v, and $\Pi^{-1} : \mathbb{R}^3 \to \mathbb{R}^2$ is a de-homogenization function. T denotes the transpose operator. Note that d_r is eventually estimated by our network based on opacity value. Eqn.6 establishes the relationship between motion and kernel locations. Next, we estimate kernel weights at each location to determine its contribution to the final blurred intensity.

3.2. Kernel Weight Estimation (Photometry)

The extent of the blur kernel is determined by the motion and depth of 3D points, while its severity depends on each point influence within the 2D kernel. For a given pose, the speed of each 3D point dictates the number of overlapping points mapping to the same location, $\underline{y}_r^t$, which affects the kernel overall size and weight distribution.This influence corresponds to the amount of time each interfering 3D point $\underline{o}_j^t$, which maps to the location $\underline{y}_r^t$, spends around the latent sharp point $\underline{o}_r^t$ while in motion. Geometrically, this represents the time $\underline{o}_j^t$ spends at the position $\underline{o}_r^t$ as it moves during the exposure period. In a blur-free scenario, $\underline{o}_r^t$ would map directly and uniquely to the latent sharp pixel $\underline{y}_r^t$. Intuitively, the time that any scene point spends at a particular location $\underline{o}_r^t$ is inversely related to its speed: slower speeds imply more time spent at a specific location and result in less blur as fewer scene points map to the same pixel, and vice-versa. In the extreme case of negligible speed (static object), there is no blur.
We carefully design a motion and depth aware 3-layer MLP to estimate weights, $\underline{w} = \{w_j\}, j = 1, 2, \cdots 2m+1$ such that $\sum_{j=1}^{2m+1} w_j = 1$; $w_j \in [0, 1]$ correspond to each kernel point. We refer to this network as BlurKernelNet. Specifically, BlurKernelNet takes 4D input, motion $\underline{\mathrm{v}}(t) \in \mathbb{R}^3$ concatenated with the depth $d(\underline{y}_r^t)$ to estimate the $(2m+1)$weights. In practice, the kernel size is seldom known apriori. The designed BlurKernelNet is agnostic to the kernel size as discussed in supplementary which also contains a description of BlurKernelNet architecture.
The training and evaluation of the network, including HexPlane, BlurKernelNet, and parameter velocities, are performed in an integrated manner.

3.3. Optimization

In our framework, the unknowns are the 3D scene radiance field, $c_i \in \mathbb{R}^3$ and $\sigma_i \in \mathbb{R}$, L ($L \ll M$) 3D control velocities $\underline{\mathrm{v}}(t) \in \mathbb{R}^3$ at time t, blur kernel consisting of continuous 2D locations $\underline{y}_j^t \in \mathbb{R}^2$ and its weights $\underline{w} \in \mathbb{R}^{2m+1}$, and the camera exposure τ. M represents the total number of time instances, i.e., the number of video frames.We assume no prior information about the exposure, hence we estimate the product $\tau\underline{\mathrm{v}}(t)$. We jointly optimize all these unknowns. Next, we discuss each loss in our optimization framework.

Photometric loss: To learn the sharp scene we optimize the radiance field by supervising the estimated color $v^{\text{blur}}(\underline{y}_r^t)$ with the observed blurry intensity $c(\underline{y}_r^t)$ at the corresponding pixel location as

$$\mathcal{L}_{\textbf{photo}} = \frac{1}{B}\sum_{r=1}^{B}\left\| v^{\text{blur}}(\underline{y}_r^t) - c(\underline{y}_r^t)\right\|^2 \tag{7}$$

where B is the batch size. We estimate $\underline{y}_j^t$ using Eqn. 5 along with the kernel weights $\underline{w}$. To prevent the degenerate case of $v^{blur}(\underline{y}_r^t) = v^{sharp}(\underline{y}_r^t)$, which can occur when $w_{m+1} \approx 1$ and $w_j \approx 0; j \neq m+1$, we used the kernel weight derivative as a regularizer i.e.,

$$\mathcal{L}_{\text{kernel}} = \|\nabla \underline{w}\|^2, \quad \text{where} \quad \nabla \underline{w} = (w_{j+1} - w_j)^2 \tag{8}$$

Eqns. 6, 7 and 8 constrain the photometric part of the scene but do not guarantee faithful learning of object motion $\underline{\text{v}}(t)$. Hence, we introduce regularizers to help the network learn a blur consistent motion.

3D direction regularizer: This ensures a smooth transition between control velocities based on the observation that most motion transitions are naturally smooth.

$$\mathcal{L}_{\text{3dd}} = \frac{1}{L-1}\sum_{t=1}^{L-1}\left(\frac{\underline{\text{v}}(t)}{||\underline{\text{v}}(t)||} \odot \frac{\underline{\text{v}}(t+1)}{||\underline{\text{v}}(t+1)||} - 1\right)^2 \tag{9}$$

Here, $\odot$ represents the dot product and $||\cdot||$ is magnitude.

3D velocity magnitude regularizer: This loss implicitly controls the velocity magnitude by adjusting the distances of kernel locations $\underline{y}_j^t$ from the central pixel $\underline{y}_r^t$.

$$\mathcal{L}_{3dm} = \frac{1}{2mB}\sum_{r=1}^{B}\sum_{j=1}^{2m+1}\text{ReLU}\left(\left|\underline{y}_r^t - \underline{y}_j^t\right| - t_h\right) \tag{10}$$

2D velocity regularizer: This provides finer control over the 3D velocities by projecting $\underline{\text{v}}(t)$ onto the image plane and supervising it with the motion of the object in the image plane.

$$\mathcal{L}_{\text{2dv}} = |\underline{\text{v}}_{mask} - T(\underline{y}_r^t, d(\underline{y}_r^t); P^v)\underline{\text{v}}(t)| \tag{11}$$

$T(\underline{y}_r^t, d(\underline{y}_r^t; P^v)) \in \mathbb{R}^{2\times 3}$ is a transformation matrix that maps 3D motion to 2D motion, depending on both pixel position and depth. We compute the motion mask center and calculate 2D motion $\underline{\text{v}}_{mask} \in \mathbb{R}^2$ by tracking the displacement of the center across time for all views. It is important to note that since the perceived 2D motion is depth-dependent, each pixel exhibits a different velocity magnitude. As a result, the calculated motion mask center cannot be considered as actual ground truth 2D speed for every pixel within the mask region. However, Eqn.11 softly guides the network to learn a faithful motion. We also use **total variational** regularizers on the feature planes to maintain spatial-temporal continuity, akin to [2]. Thus, the overall loss is given by

$$\mathcal{L} = \mathcal{L}_{\textbf{photo}} + \lambda_1\mathcal{L}_{\text{kernel}} + \lambda_2\mathcal{L}_{\text{3dm}} + \lambda_3\mathcal{L}_{\text{3dd}} + \lambda_4\mathcal{L}_{\text{2dv}} + \lambda_5\mathcal{L}_{\text{tv}} \tag{12}$$

4. Experiments

Training: Our method adapts HexPlane [2] for estimating latent sharp colors. Following [2], we empirically set λ_5 to 0.009 for real data and 0.0005 for synthetic data.We use a learning rate of 0.002 with an exponential decay rate of 0.1 for both the density and feature planes. We adopt a coarse-to-fine scene learning scheme, dynamically increasing scene resolution from $64\times64\times64$ to $200\times200\times200$. To optimize, we utilize a small 3D voxel grid to identify empty regions based on opacity and skip points within these areas. We adaptively sample points on a ray as in [2]. In all our experiments, we fix batch size $B = 2048$. We use a modified ray sampling strategy and discuss it in supplementary. Each training iteration consists of two steps. In the first step, we select B pixel locations and estimate its color (Eqn. 6) and continuous kernel locations (Eqns. 5, 6) for foreground alone. In the second step, we estimate the kernel colors and weights using BlurKernelNet (Fig. 2). We initialize $\underline{\text{v}}$ with random values in the range 0 to 0.0005. For $\underline{\text{v}}$ and BlurKernelNet, we use learning rate of 0.00001 with an exponential decay rate of 0.1. We use Adam optimizer [12] with its default parameters. Based on extensive experiments, each blurred intensity is assumed to result from the interaction of 7 distinct 3D points; thus, we set $m = 3$ and the number of control velocities to $\frac{M}{10} + 1$. We empirically select $t_h = 25$ pixels, $\lambda_1 = 0.01$, $\lambda_2 = 0.05$, $\lambda_3 = 0.00001$, and $\lambda_4 = 0.00005$ for all the experiments.

Dataset: There are no existing multiview object-blurred video datasets. To assess effectiveness of our approach, we generated synthetic videos using Blender [6] and adapted the dataset from Deblur-NeRF [24] to our problem by incorporating a rigid dynamic object into static scenes. Poses for the views are estimated using COLMAP. The synthetic data was rendered at a resolution of 1024 × 768 and a frame rate of 120 frames per second (fps). For each scene, we produced 8 to 20 videos with cameras randomly positioned within the environment to capture different perspectives. Details on how we obtain synthetic blurry videos are in supplementary. All blurred videos consist of 300 frames.

For real data, we captured 4 videos from 10 static Hero GoPro 12 cameras with a resolution of 1280 x 720. The cameras are synchronized using a laser beam and stop-timer. All cameras were configured with similar settings, and actions were performed that resulted in motion blur from the moving object. Each video consists of 120 to 300 frames. We used Movavi software [26] to generate motion masks. The resulting mask is not perfectly aligned with the object boundaries; but this is not an issue. Compared to existing

Method	Cube									Lychee								
	Static			Dynamic			Overall			Static			Dynamic			Overall		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
HexPlane [2]	28.63	0.88	0.08	19.25	0.58	0.01	26.80	0.86	0.09	35.07	0.94	0.05	18.50	0.54	0.01	28.59	0.91	0.06
MixVoxel [48]	31.32	0.92	0.04	23.26	0.82	0.01	30.91	0.91	0.04	33.64	0.92	0.04	27.48	0.78	0.01	32.73	0.91	0.05
K-Planes [10]	27.07	0.83	0.13	25.52	0.76	0.01	26.98	0.83	0.14	34.84	0.92	0.07	24.59	0.69	0.01	32.49	0.90	0.08
4DGS [52]	25.72	0.88	0.06	25.43	0.84	0.01	25.57	0.88	0.06	39.31	0.96	0.04	26.11	0.76	0.01	35.29	0.95	0.04
DynaMoDe-NeRF	32.12	0.94	0.03	26.93	0.83	0.01	31.47	0.93	0.03	33.31	0.92	0.04	27.96	0.80	0.01	32.59	0.91	0.04
Method	**Monkey**									**Sphere**								
	Static			Dynamic			Overall			Static			Dynamic			Overall		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
HexPlane [2]	40.52	0.98	0.03	22.53	0.67	0.01	32.18	0.95	0.03	31.39	0.93	0.06	18.72	0.39	0.01	29.37	0.91	0.06
MixVoxel [48]	21.95	0.75	0.09	18.05	0.42	0.01	21.41	0.72	0.10	34.52	0.94	0.03	23.75	0.66	0.01	32.70	0.93	0.03
K-Planes [10]	36.25	0.96	0.05	26.03	0.72	0.01	33.62	0.94	0.05	31.11	0.86	0.12	23.43	0.62	0.01	30.59	0.85	0.13
4DGS [52]	44.98	0.99	0.02	28.75	0.82	0.01	38.37	0.97	0.02	29.25	0.92	0.06	23.69	0.62	0.01	28.96	0.91	0.06
DynaMoDe-NeRF	40.12	0.98	0.01	30.41	0.87	0.01	37.64	0.97	0.02	36.75	0.95	0.02	24.10	0.67	0.01	34.78	0.94	0.02

Table 1. Comparisons across videos (Cube, Lychee, Monkey, and Sphere): Static (non-moving regions), Dynamic (moving foreground), Overall (entire image). The performance of our method is consistently superior in dynamic regions.

datasets for sharp scenes, such as those used in [2], [48], and [19] where dynamic activity is confined to a local region, our dataset (both synthetic and real videos) contains significant global motion that varies over time. Because we assume that blur arises from the motion of a rigid body, any human movement, if present, is treated as part of the background and ignored in the deblurring process. It is straightforward to use off-the-shelf method to segment humans.

4.1. Results and Comparisons

Since no prior work reconstructs novel views from multiview motion-blurred videos and recovers object motion, we selected baselines addressing related tasks: HexPlane [2], KPlanes [10], GS4D [52] and MixVoxel [48] for sharp multiview videos, Deblur-NeRF [24] and DP-NeRF [16] for static scene deblurring, and DyBluRF [44] for deblurring monocular videos.

For a fair comparison with Deblur-NeRF [24] and DP-NeRF [16], we sampled one image per synchronized view (representing a single time instance) for both synthetic and real data, mimicking a static scene. We trained the network with 5 to 18 views, using one image as the test view. To compare with DyBluRF [44], which works on monocular videos, we sampled frames from each view to simulate a monocular video while maintaining temporal consistency. Due to training difficulties with 300 high-resolution frames, we reduced it to 30 frames. Our approach, along with HexPlane [2], Kplanes [10], GS4D [52] and MixVoxel [48], is video-based, and we trained these methods using the same number of videos per scene, with one video as the reference view.

Qualitative comparisons for synthetic data are shown in Fig.4 and for real data in Fig.1 at a single time instance. Our approach successfully deblurs the entire moving foreground, while others fail on both real and synthetic examples. We also show the estimated motion profile of the moving object. The cosine similarity was consistently greater than 0.81. Notably, we are the first to recover motion profile of moving rigid objects.

Given the similarity of the multiview approach with ours, we present a quantitative comparison with [2] and [48] in Table 1. We use 300 frames from a single view for inference and achieve the best performance across all three regions. Our method successfully reconstructs scenes with significant object motion, recovers object motion, and deblurs frames, consistently outperforming the baselines. Additional results on real and synthetic videos with varying blur and motion profiles are provided in supplementary.

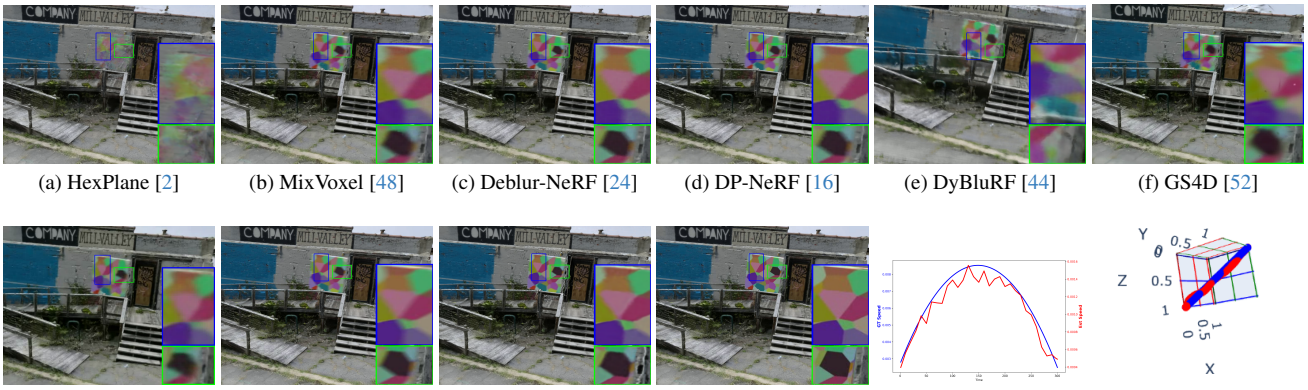
5. Ablations

The main components in our framework are multiple regularizers, number of kernel points, number of control velocities, and the interpolation type for the velocity.

Regularizers: We conduct extensive experiments with photometric loss in conjunction with other regularizers as shown in Table 2. To judge the effectiveness of regularizers, we categorized them as (i) $\boldsymbol{\mathcal{L}_{\text{photo}}} + \mathcal{L}_{\text{kernel}}$: This does not constrain the motion of the object and hence results in inconsistent and abrupt changes in the velocity profile. Hence, the need for 3D regularizers as given next. (ii) $\boldsymbol{\mathcal{L}_{\text{photo}}} + \mathcal{L}_{\text{kernel}} + \mathcal{L}_{\text{3dm}} + \mathcal{L}_{\text{3dd}}$: This loss results in color artifacts in the rendered view. This is because the 3D motion regularizer does not impose finer constraints on the velocity magnitude. (iii) $\boldsymbol{\mathcal{L}_{\text{photo}}} + \mathcal{L}_{\text{kernel}} + \mathcal{L}_{\text{2dv}}$: This renders blurry views. The total loss $\mathcal{L}$, as defined in Eqn. 12, delivers the best results. **Kernel size**: We evaluate our approach with fixed control velocities $\left(\frac{M}{10}+1\right)$, linear interpolation, and total loss $\mathcal{L}$ across empirically chosen kernel sizes: 5,

Exp Setting	Static			Dynamic			Overall		
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	LPIPS ↓
$\boldsymbol{\mathcal{L}_{\text{photo}}} + \mathcal{L}_{\text{kernel}}$	36.03	0.95	0.03	26.08	0.81	0.01	34.70	0.94	0.03
$\boldsymbol{\mathcal{L}_{\text{photo}}} + \mathcal{L}_{\text{kernel}} + \mathcal{L}_{\text{3dv}}$	35.56	0.95	0.03	20.41	0.50	0.01	30.75	0.92	0.03
$\boldsymbol{\mathcal{L}_{\text{photo}}} + \mathcal{L}_{\text{kernel}} + \mathcal{L}_{\text{2dv}}$	35.75	0.94	0.03	25.55	0.80	0.01	34.40	0.93	0.03
$\mathcal{L}$	35.58	0.95	0.02	27.35	0.79	0.01	34.12	0.94	0.02

Table 2. Effect of loss functions on test views. ($\mathcal{L}_{\text{3dv}} = \mathcal{L}_{\text{3dm}} + \mathcal{L}_{\text{3dd}}$)



(a) HexPlane [2] (b) MixVoxel [48] (c) Deblur-NeRF [24] (d) DP-NeRF [16] (e) DyBluRF [44] (f) GS4D [52]

(g) KPlanes [10] (h) Blur GT (i) Ours (j) Sharp GT (k) Speed (CS: 0.99) (l) Trajectory (CS:0.99)

Figure 4. We present and compare results on an unseen view for a single time instance (Cube video). The *Blur GT* is an unused blurred image shown for qualitative validation of the estimated sharp novel view alongside its blurry and sharp ground truth counterparts. We compare the estimated $\underline{v}\tau$ (in red) with the ground truth speed and 3D trajectory (in blue), achieving a cosine similarity (CS) of over 0.98 for complex speeds and above 0.81 for complex trajectories.

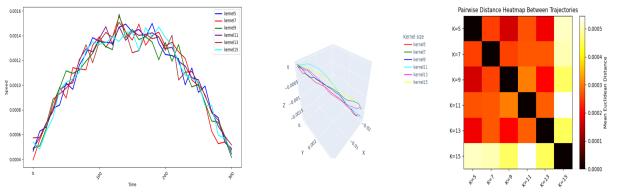


Figure 5. Effect of kernel size on 3D motion: The 3D motion profile is negligibly affected by the kernel size, showcasing the effectiveness of our approach. Left (speed profile), center (3D trajectory), and right (pairwise L2 error between the trajectories). Note that our approach can reconstruct and render sharp scenes for a long-duration video affected by time-varying blur.

7, 9, 11, 13, and 15. For each size, we trained on all videos and report PSNR, SSIM, LPIPS [58], and, speed and trajectory similarity to the ground truth in the supplementary. We observe minimal variation in rendered view quality or motion profile. Kernel size does not significantly affect performance, as discussed in the supplementary, and speed and trajectory values remain consistent, as shown in Fig. 5.

Number of control velocities: A few control velocities are sufficient for simple uniform speed and motion direction; however, motion is often unknown. We empirically select the number of control velocities to be M, $\frac{M}{10}+1$ and $\frac{M}{30}+1$. M is a trivial solution that does not affect overall visual quality of novel views. However, the estimated speed and trajectory profile is noisy. $\frac{M}{30}+1$ was too coarse and failed to estimate the correct motion profile. $\frac{M}{10}+1$ worked well consistently for all our data, (synthetic and real).

Velocity Interpolation: We experimented with linear and cubic Bezier interpolation. With respect to the rendered novel view quality, both produce similar results; however, Beizer produced a smoother speed and trajectory profile.

5.1. Limitations

Our method requires an initial rough motion mask of the moving foreground. We assume the moving body to be rigid. We have considered translational motion as it is predominant; incorporating rotational motion would be an interesting extension. Handling very severe blur remains a challenge as in any deblurring work.

6. Conclusions

We proposed a new NeRF-based approach to generate spatio-temporal novel views from multi-view object motion corrupted videos. Our method leverages the physical motion blur phenomenon, using motion as a global cue, to analytically associate blur kernel locations across different views. This enables our approach to handle time-varying blur, from minimal to significant. The proposed BlurKernelNet is agnostic to kernel size; a hyperparameter typically requiring tuning in other methods, ensuring that the recovered motion profile and rendered novel views remain consistent across various kernel sizes. Additionally, we introduced 3D regularizers that facilitate network optimization for the radiance field as well as motion in an end-to-end manner. Extensive experiments on both synthetic and real datasets demonstrate the superiority of our approach.

7. Acknowledgement

Support from Institute of Eminence (IoE) project No. SB22231269EEETWO005001 for Research Centre in Computer Vision and IoE project No. SP22231231CPETWOSSAHOC for Center of Excellence in Sports Science and Analytics is gratefully acknowledged.

References

[1] Benjamin Attal, Eliot Laidlaw, Aaron Gokaslan, Changil Kim, Christian Richardt, James Tompkin, and Matthew O'Toole. Törf: Time-of-flight radiance fields for dynamic scene view synthesis. *Advances in neural information processing systems*, 34, 2021. 3

[2] Ang Cao and Justin Johnson. Hexplane: A fast representation for dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 130–141, 2023. 1, 2, 3, 4, 6, 7, 8

[3] Mingdeng Cao, Yanbo Fan, Yong Zhang, Jue Wang, and Yujiu Yang. Vdtr: Video deblurring with transformer. *IEEE Transactions on Circuits and Systems for Video Technology*, 33(1):160–171, 2023. 3

[4] Wenbo Chen and Ligang Liu. Deblur-gs: 3d gaussian splatting from camera motion blurred images. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 7 (1):1–15, 2024. 1

[5] Sunghyun Cho and Seungyong Lee. Fast motion deblurring. *ACM Trans. Graph.*, 28(5):1–8, 2009. 3

[6] Blender Online Community. *Blender - a 3D modelling and rendering package*. Blender Foundation, Stichting Blender Foundation, Amsterdam, 2018. 6

[7] Peng Dai, Yinda Zhang, Xin Yu, Xiaoyang Lyu, and Xiaojuan Qi. Hybrid neural rendering for large-scale scenes with motion blur. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 154–164, 2023. 3

[8] Jiemin Fang, Taoran Yi, Xinggang Wang, Lingxi Xie, Xiaopeng Zhang, Wenyu Liu, Matthias Nießner, and Qi Tian. Fast dynamic radiance fields with time-aware neural voxels. In *SIGGRAPH Asia 2022 Conference Papers*, New York, NY, USA, 2022. Association for Computing Machinery. 1, 3

[9] Rob Fergus, Barun Singh, Aaron Hertzmann, Sam T. Roweis, and William T. Freeman. Removing camera shake from a single photograph. *ACM Trans. Graph.*, 25(3): 787–794, 2006. 3

[10] Sara Fridovich-Keil, Giacomo Meanti, Frederik Rahbæk Warburg, Benjamin Recht, and Angjoo Kanazawa. K-planes: Explicit radiance fields in space, time, and appearance. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12479–12488, 2023. 2, 7, 8

[11] Xiang Guo, Guanying CHEN, Yuchao Dai, Xiaoqing Ye, Jiadai Sun, Xiao Tan, and Errui Ding. Neural deformable voxel grid for fast optimization of dynamic view synthesis. In *Proceedings of the Asian Conference on Computer Vision (ACCV)*, pages 3757–3775, 2022. 3

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. 6

[13] Dilip Krishnan and Rob Fergus. Fast image deconvolution using hyper-laplacian priors. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2009. 3

[14] Orest Kupyn, Volodymyr Budzan, Mykola Mykhailych, Dmytro Mishkin, and Jiří Matas. Deblurgan: Blind motion deblurring using conditional adversarial networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 3

[15] Byeonghyeon Lee, Howoong Lee, Usman Ali, and Eunbyung Park. Sharp-nerf: Grid-based fast deblurring neural radiance fields using sharpness prior. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 3709–3718, 2024. 1, 3

[16] Dogyoon Lee, Minhyeok Lee, Chajin Shin, and Sangyoun Lee. Dp-nerf: Deblurred neural radiance field with physical scene priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12386–12396, 2023. 1, 2, 3, 7, 8

[17] Junyong Lee, Hyeongseok Son, Jaesung Rim, Sunghyun Cho, and Seungyong Lee. Iterative filter adaptive network for single image defocus deblurring. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2034–2042, 2021. 3

[18] Anat Levin, Yair Weiss, Fredo Durand, and William T. Freeman. Understanding and evaluating blind deconvolution algorithms. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1964–1971, 2009. 3

[19] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6498–6508, 2021. 1, 3, 7

[20] Zhengqi Li, Qianqian Wang, Forrester Cole, Richard Tucker, and Noah Snavely. Dynibar: Neural dynamic image-based rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4273–4284, 2023. 1, 3

[21] Jia-Wei Liu, Yan-Pei Cao, Weijia Mao, Wenqiao Zhang, David Junhao Zhang, Jussi Keppo, Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Devrf: Fast deformable voxel radiance fields for dynamic scenes. In *Advances in Neural Information Processing Systems*, pages 36762–36775. Curran Associates, Inc., 2022. 3

[22] Jia-Wei Liu, Yan-Pei Cao, Weijia Mao, Wenqiao Zhang, David Junhao Zhang, Jussi Keppo, Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Devrf: Fast deformable voxel radiance fields for dynamic scenes. *arXiv preprint arXiv:2205.15723*, 2022. 3

[23] Yu-Lun Liu, Chen Gao, Andreas Meuleman, Hung-Yu Tseng, Ayush Saraf, Changil Kim, Yung-Yu Chuang, Johannes Kopf, and Jia-Bin Huang. Robust dynamic radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 3

[24] Li Ma, Xiaoyu Li, Jing Liao, Qi Zhang, Xuan Wang, Jue Wang, and Pedro V. Sander. Deblur-nerf: Neural radiance fields from blurry images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12861–12870, 2022. 1, 2, 3, 6, 7, 8

[25] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *Computer Vision – ECCV 2020*, pages 405–421, Cham, 2020. Springer International Publishing. 1, 3

[26] Movavi. Movavi video editing software. https://www.movavi.com/. Accessed: 2024-10-21. 6

[27] Seungjun Nah, Tae Hyun Kim, and Kyoung Mu Lee. Deep multi-scale convolutional neural network for dynamic scene deblurring. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 3

[28] Jinshan Pan, Haoran Bai, and Jinhui Tang. Cascaded deep video deblurring using temporal sharpness prior. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 3

[29] Chandramouli Paramanand and Ambasamudram N. Rajagopalan. Non-uniform motion deblurring for bilayer scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2013. 3

[30] Keunhong Park, Utkarsh Sinha, Jonathan T. Barron, Sofien Bouaziz, Dan B Goldman, Steven M. Seitz, and Ricardo Martin-Brualla. Nerfies: Deformable neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5865–5874, 2021. 1, 3

[31] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T. Barron, Sofien Bouaziz, Dan B Goldman, Ricardo Martin-Brualla, and Steven M. Seitz. Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields. *ACM Trans. Graph.*, 40(6), 2021. 1, 3

[32] Cheng Peng and Rama Chellappa. Pdrf: Progressively deblurring radiance field for fast scene reconstruction from blurry images. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(2):2029–2037, 2023. 1, 3

[33] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-nerf: Neural radiance fields for dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10318–10327, 2021. 1, 3

[34] Christian Reiser, Rick Szeliski, Dor Verbin, Pratul Srinivasan, Ben Mildenhall, Andreas Geiger, Jon Barron, and Peter Hedman. Merf: Memory-efficient radiance fields for real-time view synthesis in unbounded scenes. *ACM Trans. Graph.*, 42(4), 2023. 3

[35] Barbara Roessle, Jonathan T. Barron, Ben Mildenhall, Pratul P. Srinivasan, and Matthias Nießner. Dense depth priors for neural radiance fields from sparse input views. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12892–12901, 2022. 1

[36] Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016. 3

[37] Johannes Lutz Schönberger, Enliang Zheng, Marc Pollefeys, and Jan-Michael Frahm. Pixelwise view selection for unstructured multi-view stereo. In *European Conference on Computer Vision (ECCV)*, 2016. 3

[38] Otto Seiskari, Jerry Ylilammi, Valtteri Kaatrasalo, Pekka Rantalankila, Matias Turkulainen, Juho Kannala, Esa Rahtu, and Arno Solin. Gaussian splatting on the move: Blur and rolling shutter compensation for natural camera motion. *arXiv preprint arXiv:2403.13327*, 2024. 1

[39] Qi Shan, Jiaya Jia, and Aseem Agarwala. High-quality motion deblurring from a single image. *ACM Trans. Graph.*, 27 (3):1–10, 2008. 3

[40] Ruizhi Shao, Zerong Zheng, Hanzhang Tu, Boning Liu, Hongwen Zhang, and Yebin Liu. Tensor4d: Efficient neural 4d decomposition for high-fidelity dynamic reconstruction and rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16632–16642, 2023. 1, 3

[41] Shuochen Su, Mauricio Delbracio, Jue Wang, Guillermo Sapiro, Wolfgang Heidrich, and Oliver Wang. Deep video deblurring for hand-held cameras. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017. 3

[42] Maitreya Suin and A. N. Rajagopalan. Gated spatio-temporal attention-guided video deblurring. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7802–7811, 2021.

[43] Maitreya Suin, Kuldeep Purohit, and A. N. Rajagopalan. Spatially-attentive patch-hierarchical network for adaptive motion deblurring. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. 3

[44] Huiqiang Sun, Xingyi Li, Liao Shen, Xinyi Ye, Ke Xian, and Zhiguo Cao. Dyblurf: Dynamic neural radiance fields from blurry monocular video. *arXiv preprint arXiv:2403.10103*, 2024. 2, 3, 7, 8

[45] Matthew Tancik, Vincent Casser, Xinchen Yan, Sabeek Pradhan, Ben Mildenhall, Pratul P. Srinivasan, Jonathan T. Barron, and Henrik Kretzschmar. Block-nerf: Scalable large scene neural view synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8248–8258, 2022. 1

[46] Edgar Tretschk, Ayush Tewari, Vladislav Golyanik, Michael Zollhöfer, Christoph Lassner, and Christian Theobalt. Non-rigid neural radiance fields: Reconstruction and novel view synthesis of a dynamic scene from monocular video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12959–12970, 2021. 1, 3

[47] Dor Verbin, Peter Hedman, Ben Mildenhall, Todd Zickler, Jonathan T. Barron, and Pratul P. Srinivasan. Ref-nerf: Structured view-dependent appearance for neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5491–5500, 2022. 1

[48] Feng Wang, Sinan Tan, Xinghang Li, Zeyue Tian, Yafei Song, and Huaping Liu. Mixed neural voxels for fast multi-view video synthesis. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19706–19716, 2023. 2, 3, 7, 8

[49] Peng Wang, Lingzhe Zhao, Ruijie Ma, and Peidong Liu. Bad-nerf: Bundle adjusted deblur neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4170–4179, 2023. 1, 3

[50] Yusheng Wang, Yunfan Lu, Ye Gao, Lin Wang, Zhihang Zhong, Yinqiang Zheng, and Atsushi Yamashita. Efficient video deblurring guided by motion magnitude. In *Computer*

Vision – ECCV 2022, pages 413–429, Cham, 2022. Springer Nature Switzerland. 3

[51] Silvan Weder, Guillermo Garcia-Hernando, Áron Monszpart, Marc Pollefeys, Gabriel J. Brostow, Michael Firman, and Sara Vicente. Removing objects from neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16528–16538, 2023. 1

[52] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4d gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20310–20320, 2024. 2, 7, 8

[53] Junru Wu, Xiang Yu, Ding Liu, Manmohan Chandraker, and Zhangyang Wang. David: Dual-attentional video deblurring. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 2376–2385, 2020. 3

[54] Xinguang Xiang, Hao Wei, and Jinshan Pan. Deep video deblurring using sharpness features from exemplars. *IEEE Transactions on Image Processing*, 29:8976–8987, 2020. 3

[55] Li Xu and Jiaya Jia. Two-phase kernel estimation for robust motion deblurring. In *Computer Vision – ECCV 2010*, pages 157–170, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg. 3

[56] Alex Yu, Vickie Ye, Matthew Tancik, and Angjoo Kanazawa. pixelnerf: Neural radiance fields from one or few images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4578–4587, 2021. 1

[57] Zhengming Yu, Wei Cheng, Xian Liu, Wayne Wu, and Kwan-Yee Lin. Monohuman: Animatable human neural field from monocular video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16943–16953, 2023. 1

[58] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. 8

[59] Lingzhe Zhao, Peng Wang, and Peidong Liu. Bad-gaussians: Bundle adjusted deblur gaussian splatting. *arXiv preprint arXiv:2403.11831*, 2024. 1